D7. Integrated RAG-Based Audit Assessment Chatbot

Functional prototype description and verification record.

Field	Value
Project	Development of AI-Based Cyber Security Audit Assessment Against Benchmark
Scope	Objective 3; secure local/air-gapped RAG proof of concept
Report date	28 August 2026
Evidence rule	Implemented, tested and untested claims are labelled separately.

Prototype architecture

Stage	Responsible module	What it contributes
Parse	docling_parser.py	Preserves report text and provenance paths
Retrieve	chunking.py + embed_index.py	Builds chunks and hybrid evidence candidates
Assess	pipeline.py + prompts.py	Routes question and obtains structured local answer
Guard	adaptive_rag.py	Validates citation/evidence controls and source safety
Score	evaluation_harness.py	Records results and creates audit decision

Why this is a genuine integrated prototype

The demonstration is not a disconnected collection of utilities. One run starts from supplied JSON reports, generates evidence-bearing question responses, calculates retrieval and answer measures, and writes audit-level PASS/FAIL outputs. The two-report run completed all 16 pairs with its configuration and source hashes saved. [E4][E5]

The local-first stack is suitable for sensitive reports: Python performs parsing/indexing/evaluation, FAISS and BM25 perform retrieval, and Ollama serves the generator locally. The design can be inspected component by component if an audit decision is challenged. [E4][E8][E9]

Prototype scope

AuditAssessmentPipeline accepts an audit report, a benchmark question and configuration. It returns retrieved evidence IDs/scores, response text, structured answer, CRAG state, grounding state, review flags and configuration metadata. [E8]

The prototype is local-first: Ollama is used for generation, and the repository does not require upload of project audit data to a cloud API. [E8]

End-to-end flow

Step	Component	Output / control
1	Parser and preprocessing	Provenance-labelled text; image/figure keys excluded [E8]
2	Chunking and hybrid index	Stable chunks; dense + BM25 candidates [E8]
3	Route, retrieve and optionally reformulate	Exact/local/aggregate route and CRAG verdict [E7][E8]
4	Prompt and local generation	Schema-constrained Yes/No/Insufficient Evidence response [E8]
5	Grounding and audit output	Evidence IDs, flags, question result and aggregate score [E5][E8]

Demonstrated execution

The saved two-report run processed 16 report-question pairs and recorded 22 chunks, zero execution errors and zero review flags in the run configuration. [E4]

The 28 August offline mock test exercised parser fallback, contextual chunking, FAISS/BM25 hybrid retrieval, RRF, mocked reranking/CRAG, schema output, grounding and audit-score calculation. It passed. [E7]

Use boundary

This is a proof-of-concept chatbot, not a validated decision system. It has not demonstrated visual report understanding, broad multilingual coverage, independently reviewed labels, calibrated abstention or a production deployment. [E5][E8]

Run and trace

Offline functional check: from rag/src run python test_harness_mock.py. Saved real-run outputs are in results/production_real_reports/1787846335_de0ded86a7db/. [E4][E7]

Integrated execution evidence

Input / control	Recorded result	Why it proves integration
2 JSON audit reports	22 chunks created	Demonstrates ingestion and index construction.
16 benchmark questions	16 structured records; n_errors = 0	Demonstrates complete question loop.
Local Qwen3 8B	81.25% accuracy; 13.66 s mean latency	Demonstrates local generation under saved configuration.
Evidence guard	100% grounding rate; 0 review flags	Demonstrates citation-boundary validation.
Audit aggregation	100% and 87.5% report scores	Demonstrates report-level decision output. [E4][E5]

Prototype review conclusion

The integrated result matters because all stages are connected in one run: source report identity, chunks, retrieval, prompt, model response, evidence IDs and final score are written together. A reviewer can start at a report-level PASS and inspect the corresponding question and retrieved text rather than trusting a dashboard total. [E4][E5]

This prototype does not yet establish a deployable chatbot. It requires independently labelled reports, visual-document evaluation, calibrated abstention, and stress testing before it can be used beyond a controlled POC. [E8][E10]

Engineering rationale and review use

The prototype is intentionally engineered as a sequence of inspectable components rather than a single opaque agent. That makes failures diagnosable: a reviewer can determine whether an incorrect audit decision came from parsing, chunking, retrieval, evidence selection, prompt compliance or answer generation. The saved results preserve enough intermediate data to conduct that diagnosis without replaying the entire model run. [E4][E5][E8]

This separation is particularly useful on constrained offline hardware. The two-report configuration documents which components were enabled and which were deliberately disabled to avoid GPU memory contention. The generator, retrieval settings and evidence controls can therefore be tuned independently instead of forcing a one-size-fits-all configuration that is slow, fragile or impossible to reproduce. [E4][E9]

A production extension should preserve this shape: retain the deterministic evidence and scoring layers, then strengthen the parser, labels and verification thresholds. The right next step is not a more elaborate chat UI; it is a larger independently reviewed corpus that can demonstrate whether the existing controls improve real

audit-assessment reliability. [E5][E8]

Repository evidence cited

- [E4] results/production_real_reports/1787846335_de0ded86a7db/run_config.json and run.log: two-report production-parser run configuration and execution log.
- [E5] results/production_real_reports/1787846335_de0ded86a7db/{retrieval_metrics,question_level_llm_metrics,sota_faithfulness_metrics,audit_level_scores}.json: scored metrics and audit decisions.
- [E7] Verification executed 28 August 2026: python -m unittest test_json_parser.py test_adaptive_rag.py test_security_gates.py (6 tests passed) and python test_harness_mock.py (offline mock end-to-end passed).
- [E8] src/{docling_parser.py,chunking.py,embed_index.py,pipeline.py,evaluation_harness.py,adaptive_rag.py,prompts.py,icl.py}: implementation inspection.